



01 | The Compiler Mindset: What Svelte Really Is, And How To Run It

Your code companion to the audio lecture of the same name. When the voice says "look at your booklet," this is the page. Read them together.

How To Read This Booklet

- □ — a major section. It matches a section of the audio lecture.
- ► Block N — a numbered code block. The lecture's "first / second / next" counts these inside a section.
- `title=" ... "` on a code block — the file the code lives in (or "Terminal" for commands).
- ◆ Source — where the code comes from: a docs file, or marked reconstructed / for comparison.
- 🔑 — the key line, or the move the block turns on.
- ➔ — what to notice, and why it matters.
- ✓ Works / ✗ Trap — the right way and the broken way, side by side.
- ⚠ — a surprise, or a rare-but-real gotcha.

Sources for this booklet: `/documentation 2026 June/svelte-docs/01-introduction/01-overview.md` and `... /02-getting-started.md`

Heads up: this is a mostly conceptual lecture. Many sections carry no code, just a one-line marker so you can follow along. Blocks marked REACT or VUE are background the lecture talks through, not Svelte docs code.

• • •

□ 1.1 Welcome To The Course, And To The One Idea That Changes Everything

No code yet, just the map. Six stages ahead: the mental model, the runes, template syntax, the look-and-feel layer, the runtime tools, then practice and migration. Today has two jobs: the one big idea, and a real app running on your disk. Keep this booklet open.

• • •

□ 1.2 The One Thing Nobody Tells You About A Framework: You Ship It To A Stranger

Where does a framework run? In your user's browser. React and Vue ship themselves to every visitor, in full, and run live on that stranger's three-year-old phone. Download size, plus processor time, on every single visit. That's the cost. Hold it.

• • •

□ 1.3 A Fair, Honest Refresher On How React Works

React keeps a pretend copy of the page, the virtual DOM. Every change, it rebuilds that copy and diffs it against the last one to find what moved. Diffing isn't free. Most of that walking finds that nothing changed. Rename "Alice" to "Bob" and React still rebuilds the node and checks every attribute before it reaches the one word that differs. Scale it to a thousand-row table and the waste is real, paid again and again, on every device.

• • •

❑ 1.4 And How Vue Works, Because It Chose A More Surgical Path

Vue is smarter about the update. It wraps each value in a watcher called `ref`, so when something changes it skips the diff and touches only the spots that used that value. The price: a `.value` on every read and write. And notice what didn't change. Vue is still a library, still running in the browser, still shipped along with your app.

• • •

□ 1.5 The Third Answer: What If The Framework Did Its Work Before The Browser Ever Woke Up

Svelte flips the whole thing. It is not a library that runs in the browser. It is a compiler that runs on your machine, before the app ships. It reads your code ahead of time, sees exactly what can change, and writes lean JavaScript that already knows. A library is there when your code runs. A compiler was there before your code runs. That's the hinge for the entire course.

• • •

❑ 1.6 What A Compiler Actually Does To Your Friendly Little File

You write the clean, high-level component. The compiler turns it into lean, low-level JavaScript. Here's the friendly file. What runs in the browser is the compiler's rewrite of it.

► Block 1 — The whole component (the very first code block)

◆ **Source:** `01-introduction/01-overview.md`

App.svelte

```
<script>
  function greet() {
    alert('Welcome to Svelte!');
  }
</script>

<button onclick={greet}>click me</button>

<style>
  button {
    font-size: 2em;
  }
</style>
```

- **🔑 Three parts, one file.** A `<script>` with a `greet` function, a `<button>` wired to it with `onclick`, and a `<style>` that bumps the font to `2em`. Plain HTML, CSS, and JavaScript. Nothing to decode.
- **→ This is not what ships.** The compiler reads this and emits lean JavaScript that builds the button, wires the click, and adds nothing else. You write the comfortable version. It writes the efficient one.
- **⚠ See the rewrite yourself.** Paste any component into the Svelte playground and click the "JS Output" tab. That's the compiler's actual output, laid bare. Do it once and the magic turns into plain understanding.

• • •

□ 1.7 The Real Payoff: You Get To Just Use The Language

Because the compiler does the wiring, you write plain code. Same counter bump, three frameworks, three amounts of ceremony. (Preview only, the state rune gets its own lecture.)

► Block 1 — REACT (for comparison)

◆ **Source:** reconstructed from the lecture's comparison (React)

```
setCount(count + 1); // a setter, every time. Never count++.
```

- → **React: go through the setter.** Touch the value directly and the screen freezes.

► Block 2 — VUE (for comparison)

◆ **Source:** reconstructed from the lecture's comparison (Vue)

```
count.value++; // the ".value" tax, every read and write
```

- → **Vue: go through** `.value`. The wrapper sits between you and the number.

► Block 3 — SVELTE

◆ **Source:** reconstructed from the lecture (preview)

```
count++; // just a variable. The screen follows.
```

- → **Svelte: just use the language.** No setter, no `.value`. The compiler wrote the "notify everyone" code for you, at build time. The best API is no API at all.

• • •

□ 1.8 A Short History, And Why This Course Keeps Saying Svelte Five

Svelte 3 (2019) made the compiler bet. Back then you made things reactive with a plain variable and a cryptic `$:` line. Svelte 5 (2024) kept the compiler soul but rebuilt reactivity on signals and made it explicit with runes. Same soul, sharper body. When you meet old `$:` code online, it's an earlier dialect, not something broken.

• • •

□ 1.9 The Honest Tradeoffs: What The Compiler Mindset Asks Of You In Return

No free lunch. One: a build step is mandatory, because a browser can't run a `.svelte` file directly. Two: what runs is the compiler's output, not your source, so debugging leans on source maps. Three: you trust work you don't see. Look at the "JS Output" once and that trust turns into informed delegation.

• • •

❑ 1.10 Now Let Us Actually Build One: The SvelteKit Path

Time to put a real app on disk. The recommended road is SvelteKit, the official framework around Svelte, powered by the Vite build tool. Four commands.

► Block 1 — Create and run a SvelteKit app

◆ **Source:** `01-introduction/02-getting-started.md`

>_ Terminal

```
npx sv create myapp
cd myapp
npm install
npm run dev
```

- **🔑 Four lines, empty disk to running app.** `npx sv create myapp` scaffolds a project (`npx` runs Svelte's `sv` CLI once, nothing to install). `cd myapp` steps into it. `npm install` pulls the dependencies, once. `npm run dev` starts the dev server.
- **→ The daily rhythm.** `npm run dev` fires up Vite, compiles your code, and serves the app at a local address. Save a file and the browser updates almost instantly. Edit, save, see it.

• • •

□ 1.11 The Lighter Paths: The Vite Plugin, The Playground, And A Throwaway Sandbox

SvelteKit is the default, not the only door. Three lighter ways in.

► Block 1 — Svelte with Vite, no SvelteKit

◆ Source: `01-introduction/02-getting-started.md`

>_ Terminal

```
npm create vite@latest      # then pick the "svelte" option
npm run build               # outputs HTML, JS, CSS to dist/
```

- → **Leaner and more hands-on.** Just Svelte plus Vite. The trade: no built-in routing, so add a routing library yourself for anything past one screen.
- → **The playground.** Runs in your browser, nothing to install. Best for learning, quick tests, and peeking at "JS Output." Find it at `svelte.dev/playground`.
- → **A cloud sandbox.** A full SvelteKit project in your browser in seconds, great for sharing a working example. Spin one up at `sveltekit.new`.

• • •

❑ 1.12 Your Toolbelt: The Editor Extension, The Checker, And Where To Find Help

Three small things that make week one pleasant. The big one: install the official VS Code extension on day one. It understands `.svelte` files, with highlighting, autocomplete, and live error underlines before you ever run anything. Then a command, and a place to ask.

► Block 1 — Check the whole project from the command line

◆ Source: `01-introduction/02-getting-started.md`

```
npx sv check
```

- → **One sweep, every file.** `npx sv check` runs Svelte's checker across your code: type mismatches, accessibility problems, small mistakes you missed. Same brain as the editor extension, pointed at everything at once. Perfect right before you ship.
- → **Where to get unstuck.** The Svelte Discord is famously kind to newcomers (`svelte.dev/chat`), and Stack Overflow has answers under the `svelte` tag. Ask early. That's what pros do, not beginners.

• • •

□ 1.13 The Full Recap, And The Road Just Ahead

No new code. The whole hour in one line: Svelte moves the work off your user's browser and onto your build step. A library runs in the browser; a compiler ran before it. Everything else in this course flows from that one relocation. Next up: the `.svelte` file, up close.

02 | One Component, One File: The Anatomy Of A .svelte File, And The Runes That Live Inside It

Your code companion to the audio lecture of the same name. When the voice says "look at your booklet," this is the page. Read them together.

How To Read This Booklet

- □ — a major section. It matches a section of the audio lecture.
- ► Block N — a numbered code block. The lecture's "first / second / next" counts these inside a section.
- `title=" ... "` on a code block — the file the code lives in.
- ◆ Source — where the code comes from: a docs file, or marked reconstructed / for comparison.
- 🔑 — the key line, or the move the block turns on.
- ➔ — what to notice, and why it matters.
- ⚠ — a surprise, or a rare-but-real gotcha.

Sources for this booklet: `/documentation 2026 June/svelte-docs/01-introduction/03-svelte-files.md`, `... /04-svelte-js-files.md`, and `... /02-runes/01-what-are-runes.md`

Heads up: blocks marked VUE or REACT (for comparison) are background the lecture talks through, not Svelte docs code.

• • •

□ 2.1 Where We Are: From The Why To The What

No code yet, just bearings. Lecture one was the why and the how: Svelte is a compiler, here's how to run it. Today is the what. We crack open the `.svelte` file, zone by zone, then meet the runes as a family. Keep this booklet open.

• • •

❑ 2.2 The Pain Of The Scattered Component, And The Idea Of One File

A button needs three things: logic, structure, and looks. For decades those lived in three separate files, stitched together by fragile shared class names. Svelte's fix is blunt: one component, one file, everything together. Vue did this first with its `.vue` Single File Component. React went the other way and pulled the markup *into* the JavaScript with JSX. Three directions, one goal.

• • •

□ 2.3 The Three Zones: A Guided Tour Of The Skeleton

Here's the file, in outline. Three zones, all optional: script on top, markup in the middle, style at the bottom. It's built on a superset of HTML, so any plain HTML you already know just works. Below it, the same idea in Vue and React for contrast.

► Block 1 — The .svelte skeleton (every zone a file can have)

◆ **Source:** `01-introduction/03-svelte-files.md`

MyComponent.svelte

```
<script module>
    // module-level logic goes here
    // (you will rarely use this)
</script>

<script>
    // instance-level logic goes here
</script>

<!-- markup (zero or more items) goes here -->

<style>
    /* styles go here */
</style>
```

- 🔑 **Three zones, all optional.** `<script>` for logic, markup for structure, `<style>` for looks. Use one, two, or all three. A file with nothing but markup is a complete, valid component.
- → **A superset of HTML.** Paste plain HTML straight in and it works, untouched. The plain word `class` is just `class`. Your HTML instincts carry straight over.
- ⚠️ **Two script tags?** Yes. `<script module>` runs once for the whole file; plain `<script>` runs for each copy. The next two sections pull them apart.

► Block 2 — VUE (for comparison): the Single File Component

◆ **Source:** reconstructed from the lecture's comparison (Vue, not Svelte)

MyComponent.vue

```
<template>
  <!-- markup goes here -->
</template>

<script setup>
  // logic goes here
</script>

<style scoped>
  /* styles go here */
</style>
```

- → **Svelte's close cousin.** Same idea, different tags. One note for later: Vue's `scoped` is opt-in. Svelte scopes by default.

► Block 3 — REACT (for comparison): markup pulled into the JavaScript

- ◆ **Source:** reconstructed from the lecture's comparison (React, not Svelte)

MyComponent.jsx

```
function MyComponent() {
  // logic goes here
  return (
    <button className="btn">click me</button> // "className", not "class"
  );
}
```

- → **It looks like HTML but it's JavaScript.** JSX has tells: `className` instead of `class`, and one required root element. Svelte goes the other way, HTML first.

• • •

□ 2.4 The Instance Script: A Setup That Runs Once For Every Copy

Look back at the plain `<script>` zone in the skeleton (Block 1). Two ideas live there. One: every top-level variable is automatically visible to the markup, no wiring needed. Two, the big one: the instance script runs *once*, when the component is created, then never again. Contrast React, where the component function re-runs top to bottom on every single change. Svelte runs once because the compiler already wired up the updates. (Modern Vue's `<script setup>` runs once too. Here React is the odd one out.)

• • •

❑ 2.5 The Module Script: One Brain Shared By Every Copy

The other script tag, `<script module>` runs once for the whole file, shared by every copy. Watch a single counter count the instances.

► Block 1 — A shared counter across every copy

◆ **Source:** `01-introduction/03-svelte-files.md`

App.svelte

```
<script module>
  let total = 0;
</script>

<script>
  total += 1;
  console.log(`instantiated ${total} times`);
</script>
```

- 🔑 **One shared value, many copies.** `total` lives in `<script module>`, so there's exactly one of it. Each copy's instance script bumps it and logs. First copy logs 1, second logs 2, third logs 3.
- → **Timing sets the rules.** Module code runs first, before any instance exists, so the instance script can read module variables but not the reverse. You can `export` named values from here, but never `export default` — that seat belongs to the component itself.
- ⚠ **Old code alert.** Svelte 4 wrote this as `<script context="module">`. Same thing, older spelling. And honestly, you'll rarely reach for the module script.

• • •

□ 2.6 The Style Zone: Scoped To Your Component Without You Asking

The third zone. Write CSS here and it's scoped to this component automatically. No collisions, no naming systems, no extra tooling. (The full styling lecture comes later.)

► Block 1 — Scoped styles, for free

◆ Source: `01-introduction/03-svelte-files.md`

App.svelte

```
<style>
  p {
    /* this will only affect <p> elements in this component */
    color: burlywood;
  }
</style>
```

- 🔑 **Sealed inside this component.** This `p` rule paints only the paragraphs in *this* component. Paragraphs anywhere else in the app are untouched.
- → **Svelte's default beats the rest.** React ships no styling opinion at all. Vue can scope, but you opt in with `<style scoped>`. Svelte scopes by default. You'd have to go out of your way to make a style global.

• • •

❏ 2.7 TypeScript In A Single Attribute

Want types? One attribute, and you're done.

► Block 1 — Turn the script into TypeScript

◆ **Source:** `01-introduction/03-svelte-files.md` (the `lang` attribute; the body is illustrative)

App.svelte

```
<script lang="ts">
  // everything in here is now TypeScript
  let count: number = 0;
</script>
```

- **→ TypeScript, one attribute wide.** Add `lang="ts"` to the script tag and everything inside is TypeScript: checking, autocomplete, the lot. The deep TypeScript lecture comes much later.

• • •

□ 2.8 What Is A Rune, Really? Not A Function, Though It Dresses Like One

Now the heart of the language. We keep saying "rune." Here's what one actually is, and it isn't what it looks like.

► Block 1 — A rune in the wild

◆ **Source:** `02-runes/01-what-are-runes.md`

```
let message = $state('hello');
```

-  **The docs' own definition.** "rune — a letter or mark used as a mystical or magic symbol." The Svelte team picked the word on purpose.
-  **It looks like a function. It isn't.** `$state('hello')` has the shape of a function call, but `$state` is a *keyword*, like JavaScript's `typeof` or `await`. Part of the grammar, not something you call.
- **→ Three tells that it's a keyword.** You never import it (there's no package it comes from). You can't store it in a variable or pass it around (it's an instruction at a spot, not a value). And it's only legal in certain positions (the compiler keeps you honest). Compare: React's `useState` and Vue's `ref` are imported runtime functions. A rune is none of that.

• • •

❑ 2.9 So What Do Runes Actually Do, And Why Not Just Use Functions?

So why keywords instead of functions? Because a rune is a message to the *compiler*, at build time, not a function that runs later in the browser. Different plane entirely. It even explains a neat contrast: React's hooks have placement rules (the "rules of hooks") because the framework tracks their call order at runtime. Runes have placement rules because they're grammar, read at build time. Same symptom, opposite worlds. Under the hood it's all "signals", but unlike Solid, which hands you signals directly, Svelte hides them and lets you drive with the friendly rune.

• • •

□ 2.10 When Runes Escape The Component: The .svelte.js File

Here's the payoff runes unlocked. Reactivity used to be trapped inside `.svelte` files. Now it escapes, into files named `.svelte.js` (or `.svelte.ts`).

► Block 1 — A module where runes work

◆ **Source:** reconstructed from the lecture (the full cross-file pattern is lecture four)

📄 `counter.svelte.js`

```
// A normal module – except runes work in here.  
let count = $state(0);
```

- 🔑 **The filename is the switch.** That `.svelte.` tucked before `.js` tells the compiler: process this for runes. Otherwise it's an ordinary module, with imports, exports, functions, the lot.
- → **Why it matters.** Lift reactive state and logic clean out of any component and share it across the whole app, same runes, same behavior everywhere. A capability that simply did not exist before Svelte 5. (Sharing state across files has one catch, exporting-then-reassigning, which lecture four unpacks.)

• • •

□ 2.11 Meeting The Family: A Wave At Every Rune From Across The Room

Meet the family. Seven runes, each a keyword, each a message to the compiler, each solving one problem. You'll learn them one by one. This is just the introductions.

-  `$state` — reactive state. A variable that updates the screen when it changes. (React's `useState`, Vue's `ref`.) The whole next lecture.
- `$derived` — a value computed from other reactive values, kept fresh automatically. (React's `useMemo`, Vue's `computed`.)
- `$effect` — run a side effect when its reactive dependencies change. (React's `useEffect`, Vue's `watch`.) A tool of last resort, not first.
- `$props` — what a component receives from its parent. (React props, Vue's `defineProps`.)
- `$bindable` — mark a prop as two-way bindable, so data can flow back up to the parent, not just down.
- `$inspect` — a dev-only tool that logs a piece of reactive state every time it changes. Gone in production.
- `$host` — reach the host element, only when you ship a component as a native web component.

• • •

□ 2.12 The Full Recap, And The Road Into Reactivity

No new code. The whole hour: one component, one file, three zones. And runes, which are keywords (messages to the compiler), not functions. The instance script runs once. Styles scope themselves. Runes even work outside components now, in `.svelte.js` files. Next, we sit down with the first and biggest rune: `$state`.

03 | How An App Is Built: Boxes Inside Boxes, And How They Talk

Your visual and reference companion to the audio lecture of the same name. This booklet is heavy on diagrams, tables, and small illustrations rather than verbatim code, because the lecture is the map of a whole app, not a single feature. Read them together: when the voice says "look at the diagram in your booklet," it is here.

How To Read This Booklet

- □ — a major section. Its number (3.1, 3.2, ...) matches the audio lecture's section number exactly.
- ► Block N — a numbered block: a diagram, a table, or a small code illustration.
- `title=" ... "` on a code block — the file the code lives in.
- ◆ Source — where it comes from. Most blocks here are marked *illustrative* (reconstructed from the lecture's framing, since this is a synthesis lecture); the SvelteKit file layout, the `load` function, and the form action follow the SvelteKit docs.
- 🔑 — the key idea of a block. ➔ — what to notice. ⚠ — a gotcha or a clarification.

Sources for this booklet: `/documentation 2026 June/sveltekit-docs/10-getting-started/30-project-structure.md`, `... /20-core-concepts/10-routing.md`, `... /20-load.md`, `... /30-form-actions.md`, `... /50-state-management.md`, and `/documentation 2026 June/svelte-docs/06-runtime/02-context.md`.

Two pillars to carry out of this lecture: **boxes inside boxes** (how an app is built) and **a database you can see** (what an app is for — move one object, or a list of objects).

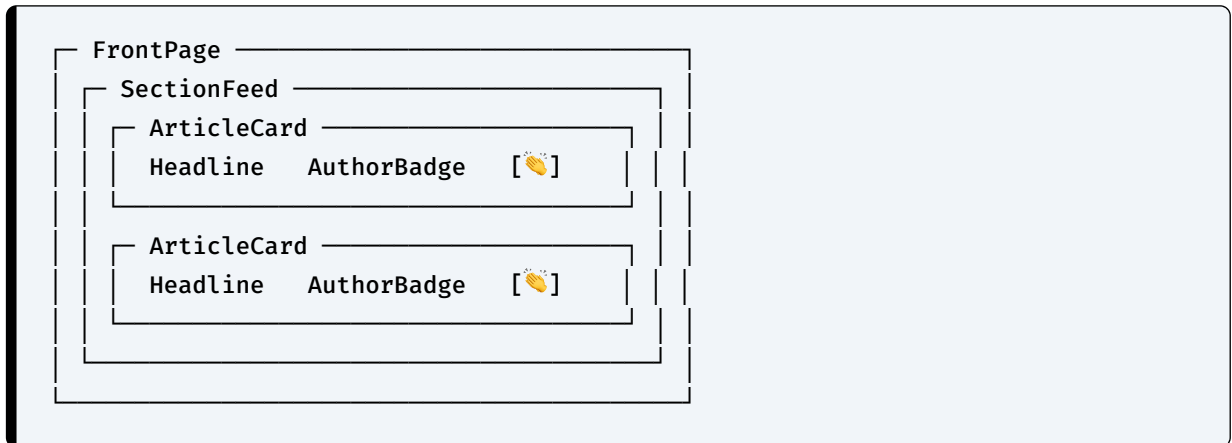
• • •

□ 3.1 The Keystone Hour, And The One Picture To Carry

The whole hour hangs on one image: an app is boxes inside boxes. A component is a box; nesting one inside another is writing a tag inside a tag.

► Block 1 — The one picture: boxes inside boxes

◆ Illustrative



- 🗝️ **A box inside a box, in code, is a tag inside a tag.** The whole page is one big box; inside it smaller boxes; inside those, smaller still, down to a single button.

• • •

❑ 3.2 Every Website Is A Database You Can See: One Object, Or A List Of Objects

Under the paint, a website is a database. A JavaScript object is a bundle of labeled values; everything that travels between parts of an app is either **one object** or **a list of objects**.

► Block 1 — The four views (every screen is one of these)

◆ Illustrative

VIEW	WHAT IT SHOWS	ON THE NATIONAL TIMES
Detail	one object	one article, full page
List	a filtered array	the front page; a section; a tag page
Edit	create / change one	the newsroom write form
Batch / live	many at once, or in place	editor dashboard; inline edit

- ➔ **Two ideas, doubled.** Present one / present a list; edit one / edit a list. Every screen you ever build is one of these four.
- ⚠ **One object, four costumes.** The same article is a card (list), a page (detail), a form (edit), a checkbox row (batch).

► Block 2 — A real front page is several filtered lists

◆ Illustrative

REGION ON THE PAGE	WHAT THE API FETCHED
Top: breaking	latest articles, top 5
Technology block	articles in section = Technology
Sports block	articles in section = Sports

- 🔑 **Each region is its own filtered list of objects, its own little API call.** The page is four or five lists, arranged in the screen real estate.

• • •

□ 3.3 The Real Shape Of An Article: Its Id, Its Slug, Its Tags, And How You Filter The Stream

The atom of the whole course. Get this object into your bones; every later lecture operates on it.

★ ► Block 1 — The article object (its real shape)

◆ **Illustrative** — the shape, in TypeScript

lib/types.ts

```
interface Article {
  id: number;           // unique, the database key
  slug: string;         // "ceasefire-talks-collapse" – the public,
  readable name
  title: string;        // "Ceasefire Talks Collapse As Fighting Resumes"
  section: string;      // "World" – the big bucket
  tags: string[];       // ["war", "ceasefire", "diplomacy"]
  authorId: number;     // 42 – a POINTER into the users table
  publishedAt: string;  // a date
  body: string;
}
```

- 🔑 **Eight labels, one parcel, one story.** id, slug, title, section, tags, author, published, body.
- → **authorId is a pointer, not the name.** It references row 42 in the users table — the relational link.

► Block 2 — The packed object the API actually returns

◆ **Illustrative** — the article *woven* with its author from another table

one article, as JSON from the server

```
{
  id: 1203,
  slug: "ceasefire-talks-collapse",
  title: "Ceasefire Talks Collapse As Fighting Resumes",
  section: "World",
  tags: ["war", "ceasefire", "diplomacy"],
  author: { id: 42, name: "Dana Reyes", articleCount: 42 },
  publishedAt: "2026-06-06T09:00:00Z",
  body: " ... "
}
```

- **🔑 Relational packing.** The server follows `authorId` into the users table and packs the real author in, plus a count of their other articles — fields from several tables, in one parcel.

► Block 3 — Stored as rows in tables (relational)

◆ Illustrative

TABLE	ONE ROW IS	KEY COLUMNS
articles	one article	id, slug, title, section, authorId, body
users	one person	id, name, role
comments	one comment	id, articleId, authorId, body

- → **One row per object; columns are the labels.** `articles.authorId` points at `users.id`. Tags (many-per-article) live in their own join table.

► Block 4 — The id versus the slug

◆ Illustrative

ASPECT	ID	SLUG
Looks like	<code>1203</code>	<code>ceasefire-talks-collapse</code>
Unique?	yes	yes
Readable?	no	yes
Role	private (database)	public (the URL)

```

nationaltimes.com/1203                ← unique, but tells a human
nothing
nationaltimes.com/ceasefire-talks-collapse ← unique AND readable (good for
SEO)

```

- **🔑 The slug is the title, lowercased, spaces → hyphens, punctuation stripped.** The link itself carries meaning.
- **⚠️ Why bother:** search engines rank readable URLs (SEO), and humans click links they can read. If two slugs would collide, add a number or date.

► Block 5 — Tag versus section

◆ Illustrative

ASPECT	SECTION	TAG
How many	a handful, fixed	thousands, open
Per article	one	many
Example	World, Sports	war, election

► Block 6 — Every label is a filter: where the lists come from

◆ Illustrative

PAGE	FILTER BY	THE LIST IT GIVES
Front page	publishedAt	newest articles, top 5
Tech section	section	articles where section = Technology
Tag page	tags	articles carrying "war"
Author page	authorId	articles where authorId = 42

- 🔑 **Same move every time:** take the stream of article objects, narrow it by one label. The fields are the handles you slice by.

► Block 7 — The unique id is also the list key

◆ Illustrative

```
{#each articles as article (article.id)}
  <ArticleCard {article} />
{/each}
```

- ⚠ **Why the `(article.id)`:** when a list changes, the framework needs a stable, unique field per item to tell them apart. Every framework asks this; the id answers it. Full story in the lists lecture (9).

• • •

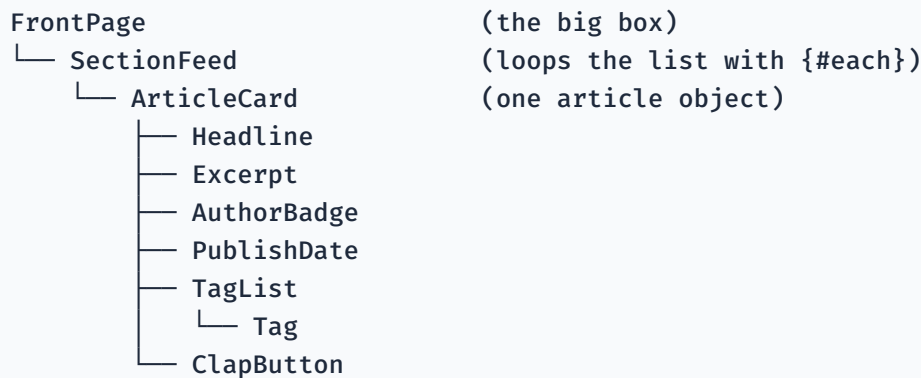
❑ 3.4 The Tree Of Boxes, And The Family That Lives In It

The structure: a tree of components. Learn the tree once, navigate it with family words.

★ ► Block 1 — The front page tree (List view)

◆ Illustrative

File Front page



► Block 2 — The article page tree (Detail view, the deep one)

◆ Illustrative

File Article page



- → The deep chain `CommentSection → CommentList → Comment → CommentBody` gives a great-great-grandchild. That depth is why the family

vocabulary matters.

► Block 3 — The family vocabulary

◆ Illustrative

RELATIONSHIP	MEANING	IN THE FRONT-PAGE TREE
parent / child	one box directly inside another	FrontPage → SectionFeed
sibling	boxes sharing a parent	Headline and ClapButton
grandparent	the parent of the parent	FrontPage is grandparent of ArticleCard
grandchild	the child of the child	ArticleCard is grandchild of FrontPage

► Block 4 — Creating a box: two moves

◆ Illustrative

📄 ClapButton.svelte (move 1: write the file)

```
<script lang="ts">
  let { onClap } = $props();
</script>

<button onclick={onClap}>👏</button>
```

📄 ArticleCard.svelte (move 2: place its tag)

```
<ClapButton onClap={clap} />
```

- 🗝️ **Make a box = write its `.svelte` file. Use a box = write its tag in a parent.** Create the file, place the tag.

• • •

❑ 3.5 The First Way Boxes Talk: Data Flows Down, Through Props

Data flows DOWN. A parent hands a child a value by writing it on the child's tag.

► Block 1 — The parent writes the tag with the value

◆ Illustrative

📄 SectionFeed.svelte (parent)

```
{#each articles as article (article.id)}
  <ArticleCard {article} />
{/each}
```

► Block 2 — The child file catches it

◆ Illustrative

📄 ArticleCard.svelte (child)

```
<script lang="ts">
  let { article } = $props(); // catches the value
</script>

<h2>{article.title}</h2>
```

- 🔑 **Parent writes the tag with the value; child catches it.** This is what lets one card show a hundred different articles.
- ⚠️ **Prop drilling:** passing a value down through many levels that don't use it, just to reach a deep grandchild. Painful at depth — cured by context (3.7). Full props lecture: 7.

• • •

□ 3.6 The Second Way Boxes Talk: Messages Flow Up, Through Events

Messages flow UP. The child never reaches up; it calls a function the parent handed down.

► Block 1 — Parent hands down a function; child calls it

◆ Illustrative

ArticleCard.svelte (parent owns the logic)

```
<script lang="ts">
  let claps = $state(0);
</script>

<ClapButton onClap={() => claps++} />
```

ClapButton.svelte (child calls back up)

```
<script lang="ts">
  let { onClap } = $props();
</script>

<button onclick={onClap}>👏</button>
```

-  **Down for data (props), up for messages (events).** The child calls the function it was given; the function runs in the parent. Full lecture: 8.

• • •

❑ 3.7 The Third Way Boxes Talk: Shared Things Flow Sideways, Through Context

Ambient things — the logged-in user, the theme — flow SIDEWAYS, reaching scattered boxes without drilling.

★ ► Block 1 — The three directions of talking

◆ Illustrative

DIRECTION	TOOL	EXAMPLE
down (parent → child)	props	FrontPage hands ArticleCard its article
up (child → parent)	events / callbacks	ClapButton tells ArticleCard
sideways (anywhere)	context / shared module	the logged-in user, everywhere

► Block 2 — Context: set high, read deep

◆ Source: `06-runtime/02-context.md` (pattern)

📄 `+layout.svelte` (high up: provides)

```
<script lang="ts">
  import { setContext } from 'svelte';
  setContext('user', user);
</script>
```

📄 `CommentForm.svelte` (deep grandchild: reads)

```
<script lang="ts">
  import { getContext } from 'svelte';
  const user = getContext('user');
</script>
```

- 🔑 **The value skips every box in between.** No drilling. Full lecture: 19. The shared-module alternative (`session.svelte.ts`) is in 3.10.

• • •

□ 3.8 What Keeps The Screen Alive: The Reactivity Toolbox, Previewed

The runes make the screen respond when data changes. The core trio, previewed.

► Block 1 — The reactivity trio

◆ Illustrative

RUNE	WHAT IT DOES	NATIONAL TIMES EXAMPLE
<code>\$state</code>	a reactive value	the clap count; menu open
<code>\$derived</code>	a value computed from others	"5 min read"; "1.2k claps"
<code>\$effect</code>	an action on change	the live ticker; autosave

- ⚠ **Without** `$state`: the count changes in memory but the screen lies.
Without `$derived`: "5 min read" goes stale. **Without** `$effect` **cleanup**: the ticker's connection leaks memory.
- → Full lectures: `$state` 4, `$derived` 5, `$effect` 6.

► Block 2 — Svelte has no hooks: the translation

◆ Illustrative

IN REACT YOU REACH FOR...	IN SVELTE IT IS...
<code>useState</code>	<code>\$state</code>
<code>useMemo</code>	<code>\$derived</code>
<code>useEffect</code>	<code>\$effect</code>

- ⚠ Make this swap once and stop hunting for hooks.

• • •

❑ 3.9 Why A Web App Is Not A Script: Reactivity, The Lifecycle, And The Asynchronous World

The biggest mental shift: a script runs once and ends; a web app stays alive, event-driven and asynchronous.

★ ► Block 1 — A plain script versus a web app

◆ Illustrative

ASPECT	A PLAIN SCRIPT	A WEB APP
Runs	top to bottom, then ends	loads, then stays alive for hours
Driven by	the sequence you wrote	events (click, type, data arriving)
Waiting	freezes until each step finishes	fires now, finishes later, never freezes

► Block 2 — The component lifecycle (why effects must clean up)

◆ Illustrative

MOMENT	TOOL	WHAT YOU DO
born (mounted)	<code>onMount</code> / <code>\$effect</code>	open the ticker connection
changes	<code>\$effect</code> re-runs	react to new state
dies (removed)	<code>onDestroy</code> / effect cleanup	close the connection — or it leaks

-  **A reactive side-effect must start at birth and be cleaned up at death.**

The `$effect` rune fuses reaction and lifecycle. Full lecture: 20.

• • •

❑ 3.10 Beyond The Boxes: The Code That Is Not A Component, And The Reusable Logic People Call Hooks

A real app is full of non-component code. Here is the whole typology.

► Block 1 — The typology of files (deploying with TypeScript)

◆ Illustrative

FILE	WHAT IT IS	EXAMPLE
<code>.svelte</code>	a component (a box)	<code>ArticleCard.svelte</code>
<code>.svelte.ts</code>	a module that can use runes	<code>session.svelte.ts</code>
<code>.ts</code>	a plain module, no runes	<code>api.ts</code> , <code>types.ts</code>
<code>+page.svelte</code>	one page / route	the front page
<code>+page.server.ts</code>	server code: <code>load</code> + <code>actions</code>	fetch / write

► Block 2 — Reusable reactive logic (Svelte's answer to custom hooks)

◆ Illustrative

lib/pagination.svelte.ts

```
export function createPagination(pageSize = 10) {
  let page = $state(1);
  return {
    get page() { return page; },
    next() { page++; },
    prev() { if (page > 1) page--; }
  };
}
```

- 🔑 **Reuse stateful logic = a function (or class) in a `.svelte.ts` file that uses runes.** Same instinct as a React custom hook, but with NONE of the rules of hooks — it is just a function.

► Block 3 — The word "hook" means three different things

◆ Illustrative

"HOOK" CAN MEAN...	IN SVELTE THAT IS...
React custom hook (reusable logic)	a function in a <code>.svelte.ts</code> file
React <code>useState</code> / <code>useEffect</code>	the runes
SvelteKit <code>hooks.server.ts</code>	server middleware (auth checkpoint)

► Block 4 — Wrapping a third-party library: an action

◆ Illustrative

in markup

```
<div use:chart={data}></div>
```

lib/chart.ts (the action)

```
export function chart(node: HTMLElement, data) {
  const c = new Chart(node, data);          // set up at birth
  return { destroy: () => c.destroy() };     // clean up at death
}
```

- **🔑 An action is the per-element lifecycle:** Svelte hands you the element, you set the library up, you return a cleanup. Full lecture: 14.

► Block 5 — Dynamic situations, and where each lives

◆ Illustrative

SITUATION	WHERE THE CODE LIVES
paginate a million articles	api module + load + pagination logic
search as you type	reusable debounce + api module
live breaking-news ticker	an <code>\$effect</code> (opens + cleans up)
log in, guard the newsroom	Supabase + server hook + load
editor autosave	an <code>\$effect</code> + a validation utility
infinite scroll	an action (intersection observer)

• • •

❑ 3.11 Where Data Enters And Leaves: Reads, Writes, FastAPI, And Supabase

Reads in `load`, writes in form `actions`, on-demand in handlers — all through one api module. FastAPI serves content; Supabase serves identity.

★ ► Block 1 — Where exactly the API calls go

◆ Illustrative

WHAT	WHERE IT GOES	EXAMPLE
page read	a <code>load</code> function	the front page's articles
write	a form <code>action</code>	publish an article
on-demand	an event handler	load more comments

- ⚠ **Not in the context** (that only shares values), and **not in a "hook"** (there are none) — reusable fetch logic is a `.svelte.ts` function.

► Block 2 — FastAPI endpoints are just "one object" or "a list"

◆ Illustrative

REQUEST	RETURNS
GET <code>/articles?sort=recent</code>	a list of objects
GET <code>/articles/{slug}</code>	one object
POST <code>/articles</code>	creates one

► Block 3 — The api module: one front door to FastAPI

◆ Illustrative

lib/api.ts

```
const BASE = 'https://api.nationaltimes.com';

export async function getLatestArticles(): Promise<Article[]> {
  const res = await fetch(`${BASE}/articles?sort=recent&limit=5`);
  return res.json(); // a list of objects
}

export async function getArticle(slug: string): Promise<Article> {
  const res = await fetch(`${BASE}/articles/${slug}`);
  return res.json(); // one object
}
```

► Block 4 — Reads: the load function feeds the props

◆ **Source:** `20-core-concepts/20-load.md` (pattern)

src/routes/+page.server.ts

```
import { getLatestArticles } from '$lib/api';

export async function load() {
  return { articles: await getLatestArticles() };
}
```

src/routes/+page.svelte

```
<script lang="ts">
  let { data } = $props(); // data.articles – then poured down through
  props
</script>
```

► Block 5 — Writes: a form action

◆ **Source:** `20-core-concepts/30-form-actions.md` (pattern)

src/routes/write/+page.server.ts

```
import { createArticle } from '$lib/api';

export const actions = {
  default: async ({ request }) => {
    const form = await request.formData();
    await createArticle(form);
  }
};
```

► Block 6 — Supabase auth: the server hook (middleware)

◆ **Illustrative**

src/hooks.server.ts

```
export async function handle({ event, resolve }) {
  const { data } = await event.locals.supabase.auth.getUser();
  event.locals.user = data.user; // who is logged in, on every request
  return resolve(event);
}
```

- ⚠️ **This is the only thing Svelte-world calls a "hook":** server middleware, the auth checkpoint. It feeds the `session.svelte.ts` that boxes read.

► Block 7 — The whole project, on disk

- ◆ **Source:** `10-getting-started/30-project-structure.md` (layout)

The National Times — src/

```
src/
├── lib/
│   ├── components/      ArticleCard.svelte, ClapButton.svelte,
│   │               CommentList.svelte
│   ├── api.ts           getLatestArticles(), getArticle(slug),
│   │               createArticle()
│   ├── types.ts         Article, User, Comment
│   ├── supabase.ts      the Supabase client
│   └── session.svelte.ts the logged-in user (reactive, shared)
├── routes/
│   ├── +layout.svelte    the masthead / shell
│   ├── +page.svelte      "/" → front page (List)
│   ├── +page.server.ts   load() → latest articles
│   ├── section/[name]/   "/section/technology" → a section (List)
│   ├── article/[slug]/   "/article/ ..." → one Article (Detail)
│   └── write/            "/write" → the Edit view
├── hooks.server.ts       runs on every request – checks the Supabase
├── session
└── app.html
```

- 🔑 **Routes folders are the addresses.** `article/[slug]` is the detail view made literal; `[slug]` catches the slug from the URL.

• • •

□ 3.12 The Whole Shape, In One Breath

(No new block — the lecture gathers everything into one breath. The table below is the whole machine on one page.)

► Block 1 — The whole app, on one page

◆ Illustrative

LAYER	WHAT IT IS
Data	a database of tables; objects and lists
Structure	components, boxes inside boxes
Talking	props down, events up, context sideways
Aliveness	<code>\$state</code> , <code>\$derived</code> , <code>\$effect</code> + the lifecycle
Plumbing	<code>load</code> (reads), <code>actions</code> (writes), one api module
Under it all	the compiler, wiring your descriptions

• • •

□ 3.13 The Reading Map: Every Tool, And Where It Lives

The capstone. Every tool previewed today, sent to the lecture that teaches it in full. After this, nothing is a surprise.

★ ► Block 1 — The reading map

◆ Illustrative

IN THE EXAMPLE	FULL LECTURE
the clap count	4 — <code>\$state</code>
"5 min read", "1.2k claps"	5 — <code>\$derived</code>
the live ticker, autosave	6 — <code>\$effect</code>
FrontPage hands ArticleCard its article	7 — <code>\$props</code>
the clap, the publish button	8 — markup & events
the section feed loop, the list key	9 — <code>{#each}</code>
the comment box, the title field	13 — <code>bind:</code>
a chart / editor library	14 — actions
the logged-in user, sideways	19 — context
born / dies, cleanup	20 — lifecycle
the article / user / comment types	23 — TypeScript
load, actions, routing, FastAPI, Supabase	the SvelteKit arc

• • •

□ 3.14 Recap, And How To Carry This Lecture

(No new block — this booklet IS the map. Keep it open. Every later lecture is a deep dive into one box of it.)

-  **Two pillars:** boxes inside boxes (how it's built), and a database you can see (what it's for — one object, or a list).
- → When a later lecture feels disorienting, come back to the reading map (3.13) and find where it sits.

04 | State Is Just A Variable: Reactivity In Svelte Five

Your code companion to the audio lecture of the same name. When the voice says "look at your booklet," this is the page. Read them together.

How To Read This Booklet

-  — a major section. It matches a section of the audio lecture.
-  Block N — a numbered code block. The lecture's "first / second / next" counts these inside a section.
-  — the headline block of a section.
- `title=" ... "` on a code block — the file the code lives in.
-  Source — where the code comes from: a docs file, or marked reconstructed / for comparison.
-  — the key line, or the move the block turns on.
-  — what to notice, and why it matters.
-  — compiler output. The "JS Output" Svelte actually generates.
-  Works /  Trap — the right way and the broken way, side by side.
-  — a surprise, or a rare-but-real gotcha.

Sources for this booklet: `/documentation 2026 June/svelte-docs/02-runes/02-$state.md`

Heads up: blocks marked REACT or VUE (for comparison), and the plain-JavaScript one, are background the lecture talks through. They are not Svelte docs code. The Svelte blocks the lecture points you to start at the counter.

This is lecture four, and it stands on lecture three, the keystone hour. The running world is the same: **The National Times**, at nationaltimes.com. The headline example is the same: the **clap count** on an article card, the little number a reader bumps by tapping the clap button. Every bare snippet in the docs here is the skeleton of something on that news site.

• • •

□ 4.1 Where We Are, And Why This Hour Matters

No code here, just bearings. Lecture three drew the whole map: boxes inside boxes, the article object, the three directions boxes talk, and a wave at the trio of runes. Now you grab the first and biggest of them, `$state`, exactly where lecture three set you down: on the clap count of an article card. Keep this booklet open.

• • •

❑ 4.2 The One Question Every Framework Is Trying To Answer

Before any framework, you wired the screen by hand. Here's the clap count built the painful way, and why it falls apart the moment a real front page has fifty live numbers on it.

► Block 1 — The naive clap count, in plain JavaScript

◆ **Source:** reconstructed from the lecture (the pain, not Svelte code)

📄 counter.js

```
let count = 0;
const display = document.getElementById('count');
display.textContent = count;

const button = document.querySelector('button');
button.onclick = () => {
  count++;
  // The card does NOT update on its own.
  // You push the new clap count back into the page by hand, every time:
  display.textContent = count;
};
```

- → **The whole problem, in one place.** `count++` changes a number in memory. The card has no idea. So you reach back in and rewrite the text yourself. One clap count is fine. Now picture the front page: a clap count and a comment count on every card, an unread badge, a most-read strip that reorders. Fifty values, each pushed by hand, each tangled with the others. That tangle is the swamp frameworks were built to drain.

• • •

□ 4.3 How React Drains The Swamp, And What It Costs You

React's deal: announce every change through a setter, and never touch the value directly. Note the word `useState` — a *hook*, the word lecture three told you to drop the second you cross into Svelte.

► Block 1 — REACT (for comparison): `useState`

◆ **Source:** reconstructed from the lecture's comparison (React, not Svelte)

Counter.jsx

```
const [count, setCount] = useState(0);

// Never mutate directly. React wouldn't notice.
setCount(count + 1); // not count++
```

- → **A setter for every value, and a rule you can't break.** `useState` hands you the value and a setter. You change the value only through the setter. Forget once, write `count++`, and the clap count freezes while the reader taps and taps. That rule, "never mutate, always call the setter," is the tax React charges.

• • •

❑ 4.4 How Vue Drains The Swamp, And What It Costs You

Vue is more surgical than React, and it's family to Svelte: both fine-grained, both updating the exact spot that changed. The price Vue asks: a `.value` on every touch, and a second tool for objects.

► Block 1 — VUE (for comparison): ref and reactive

◆ **Source:** reconstructed from the lecture's comparison (Vue, not Svelte)

Counter.vue

```
import { ref, reactive } from 'vue';

const count = ref(0);
count.value++; // the ".value" tax, on every read and write

// A second, separate tool for whole objects:
const user = reactive({ name: 'Ada' });
user.name = 'Grace';
```

- → **Two tools and a tax.** Wrap a value in `ref` and you reach it through `.value` forever, every read, every write. Whole objects need a different tool, `reactive`. Miss a `.value` once and you're poking the wrapper instead of the number.

• • •

❑ 4.5 The Big Idea: State Is Just A Variable

Here's Svelte's answer, and it sounds too simple. State is just a variable. No setter. No `.value`. You change it like any number, and the screen follows. This bare counter is the skeleton of the clap count.

★ ► Block 1 — The little counter (the very first code block)

◆ Source: `02-runes/02-$state.md`

App.svelte

```
<script>
  let count = $state(0);
</script>

<button onclick={() => count++}>
  clicks: {count}
</button>
```

- 🔑 **One line makes it reactive.** `let count = $state(0)` declares state that starts at zero. On a real card, that's the clap count, born at zero claps. That's the whole setup.
- → **Then you just use it.** A reader taps, `count++`, applause recorded. Show it with `{count}` in the markup. No setter, no `.value`. When `count` changes, that spot on the card updates itself.
- ⚠️ **Straight from the docs.** "There is no API for interacting with state. `count` is just a number, and you update it like any other variable."

Three frameworks, three deals, side by side:

ASPECT	REACT	VUE	SVELTE
Change it	<code>setCount(c + 1)</code>	<code>count.value++</code>	<code>count++</code>
The tax	a setter per value	<code>.value</code> everywhere	none
The rule	never mutate	ref vs reactive	just a variable

• • •

❑ 4.6 How A Plain Variable Can Possibly Update The Screen

How can a plain variable move the screen? One word: signal. The compiler turns your state into a smart box that holds a value and tracks who reads it. You write `count++`. The compiler writes the get, the set, and the "notify everyone" code, invisibly. You're the architect; the compiler pours the concrete.

- **→ A rune is a compiler instruction, not a function, and not a React hook.**
`$state` does not run at runtime. The compiler reads it and rewrites your plain reads and writes into signal get/set calls. That's why you never import it.
- **⚠ Want to see the rewrite?** It's shown later, in "Sharing State Across Files" (the  block). Or open the Svelte playground and click the "JS Output" tab.

• • •

□ 4.7 Deep Reactivity: When Your State Is An Object Or A List

Hand `$state` an object or array and it goes deep. Every level becomes a reactive proxy, all the way down. Picture a draft article's newsroom production checklist: the tasks that must be ticked before it goes live. Reach in, tick one, and only the matching checkbox updates.

► Block 1 — The to-do list (a deeply reactive proxy)

◆ **Source:** `02-runes/02-$state.md`

App.svelte

```
let todos = $state([
  {
    done: false,
    text: 'add more todos'
  }
]);
```

- **🔑 Deep by default.** Pass an array or object and you get a deeply reactive proxy. Read this as the checklist: each item a task, with a `done` flag and a `text`. Svelte wraps the list, then wraps every object inside it, recursively, until it hits something that isn't a plain object or array.

► Block 2 — Ticking a task (flip the done flag)

◆ **Source:** `02-runes/02-$state.md` (acts on `todos` from Block 1)

```
todos[0].done = !todos[0].done;
```

- **→ Reach in and assign. That's it.** Flip one nested property and Svelte updates only the UI tied to that exact property. Not the whole checklist. Not the whole item. Just that one checkbox.

► Block 3 — Adding a task (push a new item)

◆ **Source:** `02-runes/02-$state.md` (acts on `todos` from Block 1)

```
todos.push({
  done: false,
  text: 'eat lunch'
});
```

- **→ Plain `push` just works.** The proxy sees the push and updates the screen.

The new task gets proxied too, so it's reactive the moment it lands. In React, pushing to state is the classic frozen-screen bug. Here it's the obvious code, and it's right.

- **⚠ A footnote from the docs.** Updating a proxy doesn't mutate the original object underneath. If you bring your own proxy handlers, wrap the object first, then hand it to `$state`.

• • •

□ 4.8 The Trap That Catches Everyone: Destructuring

One move breaks reactivity, and everyone hits it. Pull a value out with destructuring and you copy it. The copy is dead. It never updates again.

► Block 1 — ✕ Trap: destructuring severs the link

◆ **Source:** `02-runes/02-$state.md` (acts on `todos` from the previous section)

```
let { done, text } = todos[0];

// this will not affect the value of `done`
todos[0].done = !todos[0].done;
```

- ✕ **Trap: the copy is frozen.** `let { done, text } = todos[0]` reads the values right now and copies them into plain variables. No live link back to the proxy. Tick the real task later and your `done` copy never hears about it.
- ✓ **The cure: reach in fresh.** Keep saying `todos[0].done` instead of tearing off a loose `done`. Don't destructure what needs to stay reactive. (This same trap bites in Vue. It's how JavaScript copies values, not a Svelte quirk.)

• • •

❑ 4.9 State Inside Classes

Classes are great for holding data with their methods. Picture a `Draft` class for the newsroom: an article that carries its own data and the methods that act on it. The docs show the seed of that idea, a tiny `Todo`. Yes, a class can hold reactive state. But here's the catch: Svelte makes plain objects reactive automatically, but not classes. With a class, you mark each reactive field yourself.

► Block 1 — The `Todo` class: reactive fields, two ways

◆ **Source:** `02-runes/02-$state.md`

📄 `todo.svelte.js`

```
class Todo {
  done = $state(false);

  constructor(text) {
    this.text = $state(text);
  }

  reset() {
    this.text = '';
    this.done = false;
  }
}
```

- **🔑 Two spots for the rune. Pick either.** Put it on the field: `done = $state(false)`. Or on the first assignment in the constructor: `this.text = $state(text)`. Both make the property reactive. Mix them freely, like this `Todo` does.
- **→ Why bother marking them?** Hand `$state` a plain object or array, and Svelte wraps it automatically. Hand it a class instance, and it does not. Wrapping a whole class could break its methods and private fields. So you opt in, one field at a time.
- **⚠️ One surprise to pocket.** The compiler turns `done` and `text` into hidden get/set methods. That makes them non-enumerable. Translation: loop over the instance's keys and they won't show up. Rare, but real.

• • •

❑ 4.10 The Other Class Trap: The Word This

A class method has a second trap, and it's pure JavaScript. Pass a method by name to a button and it loses its `this`. Picture a discard button on a draft. Here's how it breaks, and two ways to fix it.

► Block 1 — ✗ Trap: passing the method by name

◆ Source: `02-runes/02-$state.md`

App.svelte

```
<button onclick={todo.reset}>
  reset
</button>
```

- ✗ **Trap:** `this` gets hijacked. Hand `todo.reset` over bare, and when the click fires, `this` inside `reset` is the `<button>`, not the `todo`. The method runs against the wrong object. The draft never resets.

► Block 2 — ✓ Fix one: an inline arrow function

◆ Source: `02-runes/02-$state.md`

App.svelte

```
<button onclick={() => todo.reset()}>
  reset
</button>
```

- ✓ **Works: call it through the todo.** Wrap it in an arrow: `() => todo.reset()`. Now you call `reset` on `todo`, so `this` is the `todo`. Fixed. This is the same shape lecture three taught for events: the markup holds a tiny function, the click calls it.

► Block 3 — ✓ Fix two: an arrow-function field

◆ Source: `02-runes/02-$state.md`

 todo.svelte.js

```
class Todo {  
  done = $state(false);  
  
  constructor(text) {  
    this.text = $state(text);  
  }  
  
  reset = () => {  
    this.text = '';  
    this.done = false;  
  }  
}
```

- **✓ Works: bind `this` for good.** An arrow function grabs `this` from where it was born, the instance, and never lets go. Now `reset` is safe to pass around bare. Reach for this when the method gets handed off a lot, which, on a tree of boxes passing functions down, is often.

• • •

□ 4.11 When You Need Reactive Versions Of Built In Tools

`$state` makes objects and arrays reactive. It does not touch `Set`, `Map`, `Date`, or `URL`. Those are special, and the article object wants them all: a `Set` of tags (no duplicates), a `Map` keyed by article id, a `Date` for `published`, a `URL` built from the slug. Svelte ships reactive versions.

► Block 1 — Reactive Set, Map, Date, URL

◆ **Source:** `02-runes/02-$state.md` names the package (import line reconstructed; the full set gets its own lecture)

```
import { SvelteSet, SvelteMap, SvelteDate, SvelteURL } from 'svelte/
reactivity';
```

- → **Same tools, now reactive.** `Set`, `Map`, `Date`, and `URL` won't react under plain `$state`. Import the reactive twins from `svelte/reactivity` and use them exactly like the originals. Add a tag, the screen updates.

• • •

❑ 4.12 Opting Out On Purpose: The Raw Variant

Sometimes deep reactivity is more than you need. Think of a fully loaded article from your FastAPI server, body and a long comment thread, that you only ever swap whole when the reader opens a different story. `$state.raw` opts out. The deal: you can't mutate it, you can only replace it whole.

► Block 1 — The raw example (a person)

◆ **Source:** `02-runes/02-$state.md`

App.svelte

```
let person = $state.raw({
  name: 'Heraclitus',
  age: 49
});

// this will have no effect
person.age += 1;

// this will work, because we're creating a new person
person = {
  name: 'Heraclitus',
  age: 50
};
```

- ✗ **Mutate it and nothing happens.** `person.age += 1` is ignored. Raw state doesn't watch its insides.
- ✓ **Replace it whole and it works.** Assign a brand-new object to `person` and the screen updates. Raw state listens for one thing: a full swap. Fitting that the docs pick Heraclitus, who said you never step in the same river twice. Raw state is all river-swapping, no stirring.
- ⚠ **Why bother?** Big objects or arrays you'll never poke at, only swap. Skipping the deep proxy saves the cost. Raw state can still hold reactive state inside (a raw array of reactive objects), and it works on class fields too.

• • •

□ 4.13 Getting A Plain Object Back Out: The Snapshot Variant

Your state is a proxy. Great inside Svelte, confusing to the outside world, a charting library, a clone function, the console. `$state.snapshot` hands you a plain, static copy to pass along.

► Block 1 — The snapshot example

◆ Source: `02-runes/02-$state.md`

App.svelte

```
<script>
  let counter = $state({ count: 0 });

  function onclick() {
    // Will log `{ count: ... }` rather than `Proxy { ... }`
    console.log($state.snapshot(counter));
  }
</script>
```

- → **Strip the proxy, keep the data.** `$state.snapshot(counter)` returns a plain copy, no proxy attached. Perfect for handing an article to an outside tool like `structuredClone`, which chokes on a proxy.
- ⚠ **Edge case.** If the value has a `toJSON` method, the snapshot clones what `toJSON` returns instead.

• • •

❏ 4.14 A Brief Word On The Eager Variant

Usually Svelte coordinates when the screen updates, and that's better. But once in a while you want a change shown instantly, like the section nav lighting up the link the reader just clicked while the next page loads. That's `$state.eager`.

► Block 1 — The navigation bar example

◆ **Source:** `02-runes/02-$state.md`

App.svelte

```
<nav>
  <a href="/" aria-current={$state.eager(pathname) === '/' ? 'page'
: null}>home</a>
  <a href="/about" aria-current={$state.eager(pathname) === '/about'
? 'page' : null}>about</a>
</nav>
```

- **→ Instant feedback, on purpose.** `$state.eager(value)` shows a change the moment it happens, skipping the usual coordination. On The National Times, that's the section bar, World, Politics, Sports, marking the clicked link active before the page arrives.
- **⚠ Use it sparingly.** Only for feedback to a user action. The rest of the time, let Svelte time the updates. (It matters most when an `await` is in play, the async world from lecture three; full picture in the effects and runtime lectures.)

• • •

❑ 4.15 Passing State Into Functions: The Pass By Value Catch

JavaScript passes values, not variables. Hand a util the clap count and it gets a frozen copy, not a live wire. Here's the catch, and the fix.

► Block 1 — The simple add example (pass by value)

◆ Source: `02-runes/02-$state.md`

```
index.js

/**
 * @param {number} a
 * @param {number} b
 */
function add(a, b) {
  return a + b;
}

let a = 1;
let b = 2;
let total = add(a, b);
console.log(total); // 3

a = 3;
b = 4;
console.log(total); // still 3!
```

- ✕ `total` is stuck at 3. `add` got the values `1` and `2` at call time, added them, returned `3`. Later you change `a` and `b`, but `total` is just a frozen number. No link back.

► Block 2 — The second add example (pass functions)

◆ Source: `02-runes/02-$state.md`

index.js

```

/**
 * @param {() => number} getA
 * @param {() => number} getB
 */
function add(getA, getB) {
  return () => getA() + getB();
}

let a = 1;
let b = 2;
let total = add(() => a, () => b);
console.log(total()); // 3

a = 3;
b = 4;
console.log(total()); // 7

```

- **✓ Pass functions, keep the link.** Hand over `() => a` instead of `a`. A function fetches the value fresh on every call. Now `total()` sees the current numbers and returns `7`.

► Block 3 — State is no different

◆ **Source:** `02-runes/02-$state.md`

```

let a = $state(1);
let b = $state(2);

```

- **→ Same rule, same fix.** Reference `$state` and you get its current value, a plain number at that instant. Pass-by-value applies. So pass functions when you need the live clap count across a boundary.

► Block 4 — "Functions" is broad: getter properties too

◆ **Source:** `02-runes/02-$state.md`

index.js

```
/**
 * @param {{ a: number, b: number }} input
 */
function add(input) {
  return {
    get value() {
      return input.a + input.b;
    }
  };
}

let input = $state({ a: 1, b: 2 });
let total = add(input);
console.log(total.value); // 3

input.a = 3;
input.b = 4;
console.log(total.value); // 7
```

- → **Getters count as functions too.** So do a proxy's get/set. Pass the reactive `input` object and read `total.value` fresh each time. Writing a lot of this by hand? Reach for a class instead, the `Draft` from a few sections back.

• • •

❑ 4.16 Sharing State Across Files: The Export Catch

This is the sideways direction from lecture three, made real. You put the logged-in reader in a `session.svelte.js` (or `.svelte.ts`) module and share it everywhere. One rule: don't export state you also reassign. Here's why it breaks, and two clean ways around it.

► Block 1 — ✕ Trap: exporting reassigned state

◆ Source: `02-runes/02-$state.md`

📄 `state.svelte.js`

```
export let count = $state(0);

export function increment() {
  count += 1;
}
```

- ✕ **Trap: not allowed.** Export state and reassign it in the same module, and Svelte stops you. The next block shows exactly why.

► Block 2 — ⚙️ Compiler emits: why it breaks across files

◆ Source: `02-runes/02-$state.md`

📄 `state.svelte.js (compiler output)`

```
export let count = $.state(0);

export function increment() {
  $.set(count, $.get(count) + 1);
}
```

- ⚙️ **What the compiler actually writes.** Every read becomes `$.get(count)`, every write `$.set( ... )`. This is the "JS Output." Fine inside one file. But the compiler only sees one file at a time. Import `count` somewhere else and that file never gets the get/set rewrite.
- ⚠️ **See it yourself.** The "JS Output" tab in the playground shows this for any component.

► Block 3 — The leak, made visible

◆ Source: `02-runes/02-$state.md`

📄 `index.js`

```
import { count } from './state.svelte.js';

console.log(typeof count); // 'object', not 'number'
```

- → **The importing file expected a number.** It got the raw signal object instead, because it never saw the rewrite. That mismatch is the whole reason the pattern is banned.

► Block 4 — ✓ Solution one: export an object, change a property

◆ Source: `02-runes/02-$state.md`

📄 `state.svelte.js`

```
// This is allowed – since we're updating
// `counter.count` rather than `counter`,
// Svelte doesn't wrap it in `$.state`
export const counter = $state({
  count: 0
});

export function increment() {
  counter.count += 1;
}
```

- ✓ **Works: mutate, don't reassign.** You update `counter.count`, never reassign `counter`. No reassignment to rewrite, so it crosses files cleanly. This is exactly how `session` wants to be built: export the object, set `session.reader` on login, never reassign `session`. The proxy keeps it reactive everywhere, masthead greeting and deep comment form alike.

► Block 5 — ✓ Solution two: keep state private, export functions

◆ Source: `02-runes/02-$state.md`

 state.svelte.js

```
let count = $state(0);

export function getCount() {
  return count;
}

export function increment() {
  count += 1;
}
```

- **✓ Works: state stays home.** `count` never leaves the module, so every read and write stays in the one file the compiler can rewrite. Other files go through `getCount` and `increment`, the safe door through the wall. For `session`, that's `getReader`, `logIn`, `logOut`.

• • •

□ 4.17 The Full Recap, Start To Finish

No new code. Flip back through the blocks while it's fresh: the clap count, the draft checklist, the two traps, the variants, the cross-file rules. One thread ties them together. State is just a variable, because the compiler does the wiring you never see.

The four flavors of `$state`, one place to compare them:

FLAVOR	WHAT IT DOES	REACH FOR IT WHEN
<code>\$state</code>	deep reactive proxy	the default, almost always
<code>\$state.raw</code>	no deep reactivity; replace whole	big data you only swap
<code>\$state.snapshot</code>	plain static copy	handing state to an outside tool
<code>\$state.eager</code>	shows the change instantly	instant feedback on a user action

And where this rune sits on lecture three's reading map, with the next two:

IN THE NATIONAL TIMES	RUNE	LECTURE
The clap count	<code>\$state</code>	4 (here)
"5 min read", formatted count	<code>\$derived</code>	5
Autosave, live ticker	<code>\$effect</code>	6

05 | Computed Values Without Cache Bugs: The Derived Rune

Your code companion to the audio lecture of the same name. When the voice says "look at your booklet," this is the page. Read them together.

How To Read This Booklet

- □ — a major section. Its number (4.1, 4.2, ...) matches the audio lecture's section number exactly.
- ► Block N — a numbered code block. The lecture's "first / second / next" counts these inside a section.
- ★ — the headline block of a section.
- `title=" ... "` on a code block — the file the code lives in.
- ◆ Source — where the code comes from: a docs file, or marked reconstructed / for comparison.
- 🔑 — the key line, or the move the block turns on.
- ➔ — what to notice, and why it matters.
- ✓ Works / ✗ Trap — the right way and the broken way.
- ⚠ — a surprise, or a rare-but-real gotcha.

Sources for this booklet: `/documentation 2026 June/svelte-docs/02-runes/03-$derived.md`

Heads up: blocks marked REACT or VUE (for comparison), and the plain-illustration ones, are background the lecture talks through, not Svelte docs code. The Svelte source blocks the lecture points you to start at the doubled counter (4.5).

• • •

□ 5.1 Where We Are, And The Rune That Stands On State

No code here, just bearings. `$state` is your source of truth. Today's rune, `$derived`, is the value computed *from* it, kept correct forever, with no cache bugs. Next rune: `$effect`. Keep this booklet open.

• • •

❑ 5.2 The Real Problem: A Value That Lives Off Other Values

A computed value is a cache. Keep it by hand and you get one of three things: staleness, waste, or the bug-prone chore of refreshing it in every place its inputs move. Phil Karlton's joke: the two hard things in computer science are cache invalidation and naming things. This is the first one.

► Block 1 — The naive total that goes stale

◆ **Source:** reconstructed from the lecture (the pain, not docs code)

📄 Cart.svelte

```
<script>
  let items = $state([ /* ... prices ... */ ]);

  // Computed once, at creation. Then frozen.
  let total = 0;
  for (const item of items) total += item.price;
</script>

<p>Total: {total}</p>
```

- ✕ **Computed once, frozen forever.** The loop runs at creation — a component's code runs once — so `total` is a snapshot. Add an item and `total` is stale. This is exactly the bug `$derived` exists to kill.

• • •

❑ 5.3 How React Answers It, And The Cache Bug In The Bargain

React's fix is `useMemo`, with a dependency array you write and maintain by hand. That array is the cache bug, waiting to happen.

► Block 1 — REACT (for comparison): `useMemo` and the dependency array

◆ **Source:** reconstructed from the lecture's comparison (React, not Svelte)

Counter.jsx

```
const doubled = useMemo(() => count * 2, [count]);
// the [count] at the end is the dependency array – you keep it in sync by hand
```

- → **You list the dependencies yourself.** `useMemo` caches the result and recomputes only when something in `[count]` changes. Miss a dependency and the value silently goes stale.

► Block 2 — REACT (for comparison): the cache bug in the flesh

◆ **Source:** reconstructed from the lecture's comparison (React, not Svelte)

SearchList.jsx

```
const filtered = useMemo(
  () => list.filter(item => item.includes(search)),
  [list] // ✗ forgot `search` – the filter never updates when the user types
);
```

- ✗ **The bug, made concrete.** The filter reads `search`, but the array only lists `list`. Type in the box and nothing filters: React checks its array, sees `list` unchanged, and hands back the stale cache. React ships an `exhaustive-deps` linter precisely because this happens constantly.

• • •

❑ 5.4 How Vue Answers It, And How Close It Comes

Vue gets close. `computed` tracks its dependencies automatically, no array. The catch is the old one: `.value` on every read, and it's a runtime library.

► Block 1 — VUE (for comparison): `computed`

◆ **Source:** reconstructed from the lecture's comparison (Vue, not Svelte)

Counter.vue

```
import { computed } from 'vue';

const doubled = computed(() => count.value * 2); // auto-tracked, no array

console.log(doubled.value); // ...but you read it through .value
```

- → **Auto-tracked, no array.** Vue watches what the function reads, so it discovers the dependencies for you. Much closer to Svelte. But you still read it through `.value`, and all the tracking ships to every visitor at runtime.

• • •

□ 5.5 The Derived Rune: You Just Write The Expression

Svelte's answer: write the expression. That is the whole thing.

★ ► Block 1 — The doubled counter (the lecture's first code block)

◆ Source: `02-runes/03-$derived.md`

App.svelte

```
<script>
  let count = $state(0);
  let doubled = $derived(count * 2);
</script>

<button onclick={() => count++}>
  {doubled}
</button>

<p>{count} doubled is {doubled}</p>
```

- 🔑 **One line, a living formula.** `let doubled = $derived(count * 2)`. Not a value computed once — a relationship the compiler keeps true forever. Change `count`, and `doubled` follows.
- → **No array. No `.value`. No manual recompute.** The cache bug from the React section cannot happen here, because there is no array to get wrong.
- ⚠️ **Straight from the docs.** A component's code runs only once at creation, so without `$derived`, `doubled` would keep its original value even when `count` changes.
- → **On a class too.** As with `$state`, you can mark a class field `$derived`.

• • •

❑ 5.6 No Dependency Array, Ever: How The Compiler Finds Your Dependencies

How does it know the dependencies with no array? It's a compiler — it reads your expression and sees. Anything read synchronously inside is a tracked dependency.

► Block 1 — The same filter, as a derived (no bug possible)

◆ **Source:** reconstructed from the lecture (the payoff of the React bug above)

📄 SearchList.svelte

```
<script>
  let list = $state([ /* ... */ ]);
  let search = $state('');

  // You read `search` here, so it is tracked automatically.
  let filtered = $derived(list.filter(item =>
item.includes(search)));
</script>
```

- ✓ **The React bug is unwriteable here.** You read `search` in the expression, so it's a dependency, automatically. There is no array to put it in, and so none to forget it from. That is what "without cache bugs" means, concretely.

► Block 2 — Tracking reaches past an await (advanced)

◆ **Source:** `02-runes/03-$derived.md`

```
let total = $derived(await a + b);
```

- ⚠ **Advanced edge, named and deferred.** State read *after* an `await` inside the expression is tracked too. This touches the async features covered in a later lecture. (To exempt a value from being a dependency, use `untrack`.)

• • •

□ 5.7 When One Line Is Not Enough: The Derived By Form

One expression too small for the job? Use `$derived.by` and hand it a whole function.

► Block 1 — A loop inside a derived (the lecture's second code block)

◆ Source: `02-runes/03-$derived.md`

App.svelte

```
<script>
  let numbers = $state([1, 2, 3]);
  let total = $derived.by(() => {
    let total = 0;
    for (const n of numbers) {
      total += n;
    }
    return total;
  });
</script>

<button onclick={() => numbers.push(numbers.length + 1)}>
  {numbers.join(' + ')} = {total}
</button>
```

- 🔑 **A function body, when one line won't fit.** `$derived.by(() => { ... })` runs the function to get the value. `$derived(expr)` is just shorthand for `$derived.by(() => expr)`.
- → **Same tracking, either way.** The loop reads `numbers`, so `numbers` is a dependency. Push to it and `total` refreshes itself.

• • •

❑ 5.8 Lazy On Purpose: Push, Pull, And The Update That Never Happens

When does a derived recompute? Push-pull. Change state and everything that depends on it is marked dirty instantly (the push). But nothing recomputes until something reads it (the pull). And if the new value equals the old, downstream stays perfectly still.

► Block 1 — The update that never happens

◆ **Source:** `02-runes/03-$derived.md`

App.svelte

```
<script>
  let count = $state(0);
  let large = $derived(count > 10);
</script>

<button onclick={() => count++}>
  {large}
</button>
```

- **🔑 Recompute to the same answer, and nothing moves.** `large` is `count > 10`. Click nine times: `large` stays `false`, so the screen tied to it never repaints. The tenth click flips it to `true`, and it updates once.
- **→ Caching the consequences, not just the work.** A noisy input feeding a rarely-changing derived produces no downstream work until the answer truly moves. You get that for free.

► Block 2 — Stack them fearlessly (chaining)

◆ **Source:** reconstructed from the lecture (layered deriveds)

Cart.svelte

```
<script>
  let items = $state([ /* ... */ ]);

  let subtotal = $derived.by(() => items.reduce((sum, i) => sum +
i.price, 0));
  let tax      = $derived(subtotal * 0.2);
  let total    = $derived(subtotal + tax);
</script>
```

- → **A deep chain is the efficient way to build.** A derived can read another derived. Change an item and the dirty mark pushes down the whole chain at once; the pull recomputes lazily, and the identity check guards every link. Layers of small, pure formulas — correct and cheap, all the way down.

• • •

❑ 5.9 A Happy Surprise: Destructuring A Derived Is Safe

The exact opposite of the `$state` trap. Destructure a derived and every piece stays reactive.

► Block 1 — Destructuring keeps reactivity

◆ **Source:** `02-runes/03-$derived.md` (`stuff()` returns an object with `a`, `b`, `c`)

```
let { a, b, c } = $derived(stuff());
```

- ✓ **Safe, unlike `$state`**. All three stay reactive. (Destructuring `$state` severed reactivity in Lecture 4 — this does not.)

► Block 2 — Why it's safe (what Svelte does under the hood)

◆ **Source:** `02-runes/03-$derived.md`

```
let _stuff = $derived(stuff());
let a = $derived(_stuff.a);
let b = $derived(_stuff.b);
let c = $derived(_stuff.c);
```

- → **Three small deriveds, not three dead copies.** Svelte rebuilds the destructuring as one derived per property, each with its own living relationship to the source. That is why each one stays current.

• • •

□ 5.10 Deriveds Are Left As They Are, Not Wrapped

Unlike `$state`, a derived isn't wrapped in a new proxy. It hands back the real thing.

► Block 1 — A live window onto reactive state

◆ Source: `02-runes/03-$derived.md`

App.svelte

```
let items = $state([ /* ... */ ]);  
  
let index = $state(0);  
let selected = $derived(items[index]);
```

- 🔑 **Not a copy — the actual item.** `selected` is the very object inside the reactive `items` array. Change a property of `selected` (or `bind:` to it) and the underlying item updates, because it was never copied.
- ⚠️ **Only works because `items` is deeply reactive.** If `items` weren't a `$state` proxy underneath, mutating `selected` would do nothing.

• • •

❑ 5.11 Bending The Rule For Optimistic UI: Overriding A Derived

One sanctioned rule-break: reassign a derived temporarily, for instant feedback ahead of a slow server.

► Block 1 — Optimistic UI

◆ **Source:** `02-runes/03-$derived.md`

LikeButton.svelte

```
<script>
  let { post, like } = $props();

  let likes = $derived(post.likes);

  async function onclick() {
    // increment the `likes` count immediately...
    likes += 1;

    // and tell the server, which will eventually update
    `post`
    try {
      await like();
    } catch {
      // failed! roll back the change
      likes -= 1;
    }
  }
</script>

<button {onclick}>❤️ {likes}</button>
```

- **🔑 Bump now, reconcile later.** `likes += 1` immediately for instant feedback, then call the server, then roll back (`likes -= 1`) on failure. When `post` actually updates, the formula reasserts itself and takes over again.
- **⚠️ Recent power, and the exception not the rule.** Before Svelte 5.25, deriveds were read-only. Use this override only for optimistic feedback; almost every derived you write is a pure formula you never touch by hand.

• • •

□ 5.12 The One Firm Rule: Keep The Expression Pure

One firm rule: the expression must be pure. Read state, return a value, change nothing else.

► Block 1 — ✕ The compiler stops you

◆ **Source:** reconstructed from the lecture (illustrating the rule)

```
// ✕ Not allowed – Svelte disallows state changes inside a derived.  
let doubled = $derived(count++);
```

- ✕ **No side effects inside.** A state change like `count++` in a derived is disallowed, by the compiler. Why: a derived runs lazily — zero times, or many, at moments you don't control — so a side effect there would fire chaotically.
- → **Need a side effect? That's** `$effect`. The urge to *do* something inside a derived means you want an effect, not a value. That is the next lecture.

• • •

□ 5.13 The Full Recap, And The Road To Effects

No new code. Flip back through the derived blocks while they're warm: the doubled counter, the `$derived.by` total, the lazy `large`. State holds the truth, derived computes pure values from it, effect (next) touches the world. You describe what a value *is*, once, and a compiler that read your description keeps it true, fresh, and cheap — computed values, genuinely without cache bugs.